

JSON

1. What is JSON?

- a. *JSON* stands for *JavaScript Object Notation*.
- b. It is a format for structuring data.
- c. It is a lightweight data-interchange format used for storing and exchanging data.
- d. It represents data as key-value pairs and supports data structures like objects and arrays.
- e. To create a JSON file we have to use the *.json* extension.
- f. For Example → *db.json*

2. Key Terms:

- a. **Collection**: A collection of documents. (represented by [])
- b. **Document**: A collection of fields (fields are key-value pairs) (represented by {})

3. Why Use JSON?

- a. *Cross-Language Compatibility*: JSON is supported by most programming languages.
- b. *Lightweight*: It has minimal syntax, making it faster to parse and transfer.
- c. *Readability*: Easy to understand and debug for developers.
- d. *Efficient Data Representation*: Can represent complex nested data structures.

4. Where to Use JSON?

- a. *Data Exchange*: Between a client (e.g., browser) and server (e.g., API response).
- b. *Configuration Files*: Storing application settings.
- c. *Database Formats*: Many NoSQL databases (e.g., MongoDB) store data in JSON-like formats.
- d. *Frontend Frameworks*: To manage data (e.g., React, Angular, or Vue).

5. How to Create JSON?

a. Rules for JSON:

- i. Data is represented as *key-value* pairs.
- ii. *Keys* must be strings enclosed in *double quotes*.
- iii. Values can be strings, numbers, objects, arrays, booleans, or null.

b. JSON Syntax:

```
[
  {
    "name": "John Doe",
    "age": 20,
    "isStudent": true,
    "skills": ["HTML", "CSS", "Tailwind CSS", "JS"],
    "address": {
      "street": "123 Main st",
      "city": "New York",
      "zip": 10001
    }
  }
]
```

6. JSON Objects:

- a. JSON objects are written inside curly braces `{ }`.
- b. Just like in JavaScript, objects can contain multiple name/value pairs:
- c. **For Example** →

```
{ "firstName": "John", "lastName": "Doe" }
```

7. JSON Arrays:

- a. JSON arrays are written inside square brackets `[ ]`.
- b. Just like in JavaScript, an array can contain objects:
- c. **For Example** →

```
"employees": [  
  { "firstName": "John", "lastName": "Doe" },  
  { "firstName": "Anna", "lastName": "Smith" },  
  { "firstName": "Peter", "lastName": "Jones" }  
]
```

8. JSON Methods in JavaScript:

- a. There are 2 main popular JSON methods used widely:
 - i. `JSON.parse()`
 - ii. `JSON.stringify()`

9. `JSON.parse()`:

- a. `JSON.parse()` converts a JSON string to a JavaScript object.
- b. It accepts a JSON string as input.
- c. When receiving data from a web server, the data is always a string.

- d. Parse the data with `JSON.parse()` , and data becomes a Javascript Object.
- e. It's useful for storing data in local storage as browsers store data in key value pairs.
- f. **Syntax** →

 `JSON.parse(string)`

- g. **For Example** →

```
let jsonString = '{"name": "John", "age": 20}';  
let jsObject = JSON.parse(jsonString);  
console.log(jsObject);
```

10. `JSON.stringify()`:

- a. `JSON.stringify()` converts JavaScript objects into JSON strings, accepting a single object argument.
- b. When sending data to a web server, the data has to be a string format.
- c. Convert a javascript object into a string with `JSON.stringify()`.
- d. **Syntax** →

 `JSON.stringify(object)`

- e. **For Example** →

```
let jsObject = { name: "John", age: 20 };  
let jsonString = JSON.stringify(jsObject);  
console.log(jsonString);
```

11. Steps to Create a JSON Server Locally:

a. **Step-1:** To create a folder name as " **server** ".

b. **Step-2:** Create a JSON file API.

i. File Name: **db.json**

ii. This file will act as the database for the JSON server. Example db.json content:

```
{  
  "users": [  
    { "id": 1, "name": "Alice", "age": 25 },  
    { "id": 2, "name": "Bob", "age": 30 }  
  ]  
}
```

c. **Step-3:** Install **json-server** using npm.

i. Open a terminal, navigate to the **server** folder, and run the following command:

 `npm install json-server`

d. **Step-4:** Set up a **package.json** file to manage dependencies.

i. Run the following command to create a package.json file with default settings:

 `npm init --yes`

e. **Step-5:** Add a script to start the JSON server.

i. Open the package.json file and add the following under "scripts" the section:

 `"start": "json-server -- watch fileName.json --port portNumber"`

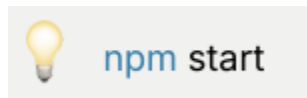
ii. For Example →

```
"scripts": {  
  "start": "json-server --watch db.json --port 8000"  
}
```

iii. This sets up the **npm start** command to launch the server.

f. **Step-6:** Start the server to host the data.

i. Run the following command to start the server.



ii. The server will be available at <http://localhost:8000>.

12. Advantages of JSON:

- a. **Lightweight and Compact:** Smaller in size compared to XML, which makes data transfer faster.
- b. **Platform Independent:** Works seamlessly across different programming languages.
- c. **Easy to Use:** Simple structure with a clear syntax.
- d. **High Performance:** Easy to parse by most modern libraries and APIs.

13. Disadvantages of JSON:

- a. **No Support for Comments:** Unlike XML or other formats, JSON does not allow comments.
- b. **Limited Data Types:** Only supports text, numbers, arrays, objects, booleans, and null.
- c. **No Schema Validation:** JSON does not have a built-in schema validation mechanism.